INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
IFCE *CAMPUS* JAGUARIBE
TECNOLOGIA EM REDES DE COMPUTADORES

JOSÉ VINÍCIUS ALVES DE OLIVEIRA

PLATAFORMA WEB COM LLM LOCAL PARA APOIO À PROGRAMAÇÃO DE SISTEMAS EMBARCADOS DIDÁTICOS

JAGUARIBE
2026

JOSÉ VINÍCIUS ALVES DE OLIVEIRA

PLATAFORMA WEB COM LLM LOCAL PARA APOIO À PROGRAMAÇÃO DE SISTEMAS EMBARCADOS DIDÁTICOS

Monografia apresentada ao curso de Tecnologia em Redes de Computadores do Instituto Federal de Educação, Ciência e Tecnologia do Ceará (IFCE) – *Campus* Jaguaribe, como requisito parcial para obtenção do Título de Tecnólogo em Redes de Computadores. Área de concentração: Ciência da Computação.

Orientador: Prof. Dr. Herleson Paiva Pontes.

JAGUARIBE
2026

Página reservada para a ficha catalográfica, que você pode elaborar por meio do Gerador de Ficha Catalográfica *on-line* do IFCE.

JOSÉ VINÍCIUS ALVES DE OLIVEIRA

PLATAFORMA WEB COM LLM LOCAL PARA APOIO À PROGRAMAÇÃO DE SISTEMAS EMBARCADOS DIDÁTICOS

Monografia apresentada ao curso de Tecnologia em Redes de Computadores do Instituto Federal de Educação, Ciência e Tecnologia do Ceará (IFCE) – *Campus* Jaguaribe, como requisito parcial para obtenção do Título de Tecnólogo em Redes de Computadores. Área de concentração: Ciência da Computação.

Aprovado (a) em: _____ / _____ / _____.

BANCA EXAMINADORA

Prof. Dr. Herleson Paiva Pontes (Orientador)
Instituto Federal de Educação, Ciência e Tecnologia do Ceará (IFCE) – *Campus* Jaguaribe

---

Prof. [Titulação] [Nome do avaliador]
[Instituição do avaliador]

Prof. [Titulação] [Nome do avaliador]
[Instituição do avaliador]

Dedico este trabalho à memória do meu irmão Edcarlos Gomes Morais, que ainda me guia e inspira.

AGRADECIMENTOS

À minha família, pelo apoio e incentivo durante toda a minha vida acadêmica. Especialmente à minha mãe e irmãos, que sempre me apoiaram nos momentos mais difíceis.

À minha companheira, Maria Eduarda de Lima Pinheiro, pelo amor, paciência e por estar em cada desafio ao meu lado.

Aos meus amigos, que sempre estiveram presentes e me ajudaram a superar os desafios e celebrar as conquistas ao longo desta jornada acadêmica.

Aos meus professores, que compartilharam seus conhecimentos e experiências, contribuindo para o meu crescimento acadêmico e pessoal. Em especial, ao meu orientador, Herleson Paiva Pontes, que me guiou em muitos outros desafios acadêmicos e profissionais, e que me ajudou a superar os desafios deste trabalho.

Aos meus colegas de curso, que fizeram parte desta jornada acadêmica e tornaram a experiência mais enriquecedora e memorável.

A todos que, de alguma forma, contribuíram para a realização deste trabalho, direta ou indiretamente. Agradeço a cada um de vocês por fazerem parte desta etapa da minha vida.

RESUMO

Este trabalho apresenta o desenvolvimento do Duckoder, uma plataforma *web* integrada a um modelo de linguagem de grande porte (LLM) executado localmente, para auxiliar estudantes na programação de placas didáticas de sistemas embarcados. A solução é composta por uma interface *web*, na qual o estudante escolhe a placa, descreve a atividade em linguagem natural e refina o código na mesma conversa; por uma API REST que insere automaticamente a descrição do *hardware* da placa em cada requisição e mantém o histórico das conversas; e por um modelo aberto executado em um servidor da rede local por meio do Ollama, dispensando serviços em nuvem. A avaliação utilizou dez atividades práticas de complexidade crescente, divididas entre o ESP32 e o Arduino Mega 2560 e submetidas três vezes a dois modelos, o Qwen2.5-Coder 7B e o Llama 3.1 8B, e considerou a taxa de códigos funcionais e o tempo de resposta. O Qwen2.5-Coder produziu código funcional na primeira resposta em 83% das gerações e em 87% após o refinamento; o Llama 3.1, em 67% e 77%, respectivamente. Todas as gerações das atividades de complexidade baixa funcionaram na primeira resposta, e as falhas concentraram-se em bibliotecas específicas. O refinamento pela conversa recuperou parte dos erros de compilação, mas nenhum código que compilava e se comportava de forma diferente da pedida. O tempo médio de resposta ficou entre 12 e 14 s, e a comunicação na rede local acrescentou apenas milissegundos. Conclui-se que um modelo aberto executado na infraestrutura da instituição é suficiente para as atividades de disciplinas introdutórias, com independência de conexão externa, sem custo por requisição e com os dados mantidos na instituição.

Palavras-chave: sistemas embarcados; modelos de linguagem de grande porte; aplicação *web*; rede local; ensino.

ABSTRACT

This work presents the development of Duckoder, a web platform integrated with a locally hosted large language model (LLM) to help students program educational embedded systems boards. The solution consists of a web interface, where the student chooses the board, describes the activity in natural language and refines the code in the same conversation; a REST API that automatically adds the board's hardware description to each request and keeps the conversation history; and an open model running on a local network server through Ollama, with no need for cloud services. The evaluation used ten hands-on activities of increasing complexity, split between the ESP32 and the Arduino Mega 2560 and submitted three times to two models, Qwen2.5-Coder 7B and Llama 3.1 8B, and considered the rate of working code and the response time. Qwen2.5-Coder produced working code in the first answer in 83% of the generations and in 87% after refinement; Llama 3.1, in 67% and 77%, respectively. All generations for the low-complexity activities worked in the first answer, and the failures were concentrated on specific libraries. Refinement through the conversation recovered part of the compilation errors, but none of the programs that compiled and behaved differently from what was requested. The mean response time was between 12 and 14 s, and communication over the local network added only milliseconds. The results show that an open model running on the institution's own infrastructure is enough for the activities of introductory courses, without depending on an external connection, with no cost per request and keeping the data within the institution.

Keywords: embedded systems; large language models; web application; local network; education.

LISTA DE FIGURAS

Figura 1 — Arquitetura proposta para a plataforma 20
Figura 2 — Interface do Duckoder ao fim de uma conversa com refinamento . . 28
Figura 3 — Painel da placa com as ligações da versão 2 do código 29

LISTA DE QUADROS

Quadro 1 — Comparação entre os trabalhos relacionados e a proposta 18
Quadro 2 — Requisitos funcionais e não funcionais da plataforma 19
Quadro 3 — Atividades práticas utilizadas na avaliação 23
Quadro 4 — Mecanismo de implementação que atende cada requisito 26
Quadro 5 — Recursos expostos pela API 26
Quadro 6 — Exemplo de requisição e resposta da API 27

LISTA DE TABELAS

Tabela 1 — Códigos funcionais por atividade, em três gerações por modelo . . . 30
Tabela 2 — Tempo de resposta por atividade, em segundos 34

LISTA DE SIGLAS

API	*Application Programming Interface*
CSS	*Cascading Style Sheets*
GPIO	*General Purpose Input/Output*
GPU	*Graphics Processing Unit*
HTML	*HyperText Markup Language*
HTTP	*Hypertext Transfer Protocol*
I2C	*Inter-Integrated Circuit*
IA	Inteligência Artificial
IDE	*Integrated Development Environment*
IFCE	Instituto Federal de Educação, Ciência e Tecnologia do Ceará
IP	*Internet Protocol*
JSON	*JavaScript Object Notation*
LED	*Light-Emitting Diode*
LLM	*Large Language Model*
OLED	*Organic Light-Emitting Diode*
PWM	*Pulse Width Modulation*
RAM	*Random Access Memory*
REST	*Representational State Transfer*
SoC	*System on Chip*
SPI	*Serial Peripheral Interface*
UART	*Universal Asynchronous Receiver/Transmitter*
URI	*Uniform Resource Identifier*
USB	*Universal Serial Bus*

SUMÁRIO

1 INTRODUÇÃO . . . 13
1.1 Objetivos . . . 13
1.1.1 Objetivo geral . . . 13
1.1.2 Objetivos específicos . . . 14
1.2 Organização do trabalho . . . 14
2 FUNDAMENTAÇÃO TEÓRICA . . . 15
2.1 Sistemas embarcados no ensino . . . 15
2.2 Modelos de linguagem de grande porte . . . 16
2.3 Aplicações *web* e APIs REST . . . 17
2.4 Trabalhos relacionados . . . 18
3 METODOLOGIA . . . 19
3.1 Requisitos e arquitetura . . . 19
3.2 Implementação da API e integração com o LLM . . . 20
3.3 Desenvolvimento da interface *web* . . . 22
3.4 Procedimento de avaliação . . . 22
4 RESULTADOS E DISCUSSÃO . . . 25
4.1 Requisitos e arquitetura . . . 25
4.2 API REST . . . 25
4.3 Interface *web* . . . 27
4.4 Avaliação da plataforma . . . 30
4.4.1 Taxa de códigos funcionais . . . 30
4.4.2 Tempo de resposta . . . 33
4.4.3 Discussão . . . 35
5 CONCLUSÃO . . . 37
REFERÊNCIAS . . . 39
APÊNDICE A — DESCRIÇÃO DO *HARDWARE* DAS PLACAS . . . 40
A.1 ESP32 DevKit V1 . . . 40
A.2 Arduino Mega 2560 . . . 41
APÊNDICE B — PEDIDOS USADOS NA AVALIAÇÃO . . . 43

1 INTRODUÇÃO

Disciplinas práticas das áreas de tecnologia e engenharia dependem de kits e plantas didáticas para aproximar teoria e prática, mas as soluções comerciais costumam ser caras e pouco flexíveis. Por isso, diversos trabalhos propõem alternativas de baixo custo para o ensino, como sistemas de aquisição e controle baseados em ESP32 (Duarte et al., 2024), simuladores de sistemas dinâmicos em tempo real usados como plantas didáticas programáveis (Souza et al., 2014) e a placa aberta BitDogLab (Fruett et al., 2024). Mesmo com o *hardware* acessível, porém, programar essas placas ainda representa uma barreira para estudantes iniciantes.

No trabalho da BitDogLab, essa barreira é enfrentada com um modelo de linguagem de grande porte (LLM, do inglês *Large Language Model*), o ChatGPT, que recebe uma descrição textual do *hardware*, indicando quais componentes estão ligados a quais pinos, e, a partir dela, gera e corrige códigos para a placa (Fruett et al., 2024). Essa abordagem, no entanto, depende de um serviço comercial em nuvem e de acesso à internet, envia as interações dos estudantes a terceiros e exige que a descrição do *hardware* seja carregada manualmente no *chat*.

Diante desse cenário, este trabalho propõe o Duckoder, uma plataforma que oferece essa assistência de forma local. A solução é composta por três partes: uma interface *web*, na qual o estudante escolhe a placa, descreve em linguagem natural o que deseja que ela faça e consulta como ligar os componentes usados pelo código; uma interface de programação de aplicações (API, do inglês *Application Programming Interface*) no estilo REST (do inglês *Representational State Transfer*), que armazena os perfis de *hardware* das placas, insere automaticamente essa descrição em cada requisição e mantém o histórico das conversas; e um LLM aberto executado em um servidor da rede local, por meio do Ollama. Assim, a plataforma funciona sem acesso à internet, sem custos com serviços externos e sem que os dados saiam da instituição.

O *hardware* é tratado apenas como alvo de validação, sem detalhamento do projeto eletrônico. Para avaliar a plataforma, utiliza-se um conjunto de atividades práticas de complexidade crescente, cujos códigos são gerados pela plataforma e executados, medindo-se a proporção de códigos que funcionam corretamente e o tempo de resposta do modelo no servidor local. A gravação do código na placa permanece a cargo do estudante, que o copia para um ambiente de desenvolvimento integrado (IDE, do inglês *Integrated Development Environment*), como faria com um código escrito por ele.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver uma plataforma *web* integrada a um LLM executado localmente, para auxiliar estudantes na programação de placas didáticas de sistemas embarcados sem depender de serviços em nuvem.

1.1.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

a) definir os requisitos e a arquitetura da plataforma, incluindo o formato da descrição de *hardware* utilizada como contexto pelo modelo;

b) implementar uma API REST que integre um LLM aberto, executado em servidor na rede local, e insira automaticamente a descrição da placa nas requisições;

c) construir uma interface *web* em que o estudante descreva a atividade em linguagem natural e receba o código gerado com explicações;

d) avaliar a plataforma em um conjunto de atividades práticas, quanto à taxa de códigos funcionais e ao tempo de resposta.

1.2 Organização do trabalho

Este trabalho está organizado em cinco capítulos. Após esta introdução, o Capítulo 2 apresenta a fundamentação teórica sobre sistemas embarcados no ensino, modelos de linguagem de grande porte e aplicações *web*. O Capítulo 3 descreve a metodologia adotada para cada objetivo específico, e o Capítulo 4 apresenta e discute os resultados obtidos. Por fim, o Capítulo 5 reúne as conclusões e as sugestões de trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos necessários para acompanhar a metodologia e os resultados. A seção 2.1 trata do uso de plataformas embarcadas de baixo custo em atividades práticas de ensino e apresenta as placas adotadas como alvo de validação. A seção 2.2 conceitua os modelos de linguagem de grande porte, seu uso na geração de código e a execução local de modelos abertos. A seção 2.3 descreve a arquitetura cliente-servidor e o estilo REST, que estruturam a comunicação entre as partes da plataforma. Por fim, a seção 2.4 compara os trabalhos da base de referências com a proposta.

2.1 Sistemas embarcados no ensino

Sistemas embarcados são sistemas computacionais dedicados a uma função específica dentro de um equipamento maior, construídos em geral sobre microcontroladores que integram processador, memória e periféricos em um único circuito integrado. Em disciplinas práticas das áreas de tecnologia e engenharia, esses sistemas atuam como plantas didáticas, nas quais o estudante programa o dispositivo, observa o comportamento físico resultante e relaciona esse comportamento ao conteúdo teórico da disciplina.

A adoção de plataformas de baixo custo nesse papel responde ao preço e à rigidez dos kits comerciais. Duarte et al. (2024) desenvolvem um sistema embarcado de aquisição e controle baseado em ESP32 para disciplinas experimentais, com custo bastante inferior ao de bancadas didáticas equivalentes. Souza et al. (2014) propõem uma plataforma de simulação em tempo real que reproduz o comportamento de sistemas dinâmicos em *hardware* embarcado, permitindo que uma única placa represente diferentes plantas; o trabalho é estendido em Souza, Silva e Fernandes (2016) para outras arquiteturas embarcadas, com suporte a simulação do tipo *hardware-in-the-loop*. Em outra direção, Parreira et al. (2021) montam um laboratório virtual de sistemas embarcados apoiado em *softwares* gratuitos, voltado ao ensino remoto. Mais recentemente, Fruett et al. (2024) apresentam a BitDogLab, uma placa aberta de baixo custo para atividades educacionais em ciência e tecnologia.

Neste trabalho, duas placas são adotadas como alvo de validação: o ESP32 e o Arduino Mega 2560. O ESP32 é um sistema em um *chip* (SoC, do inglês *System on Chip*) de baixo custo desenvolvido pela Espressif Systems, que integra até dois processadores Xtensa LX6 de 32 bits operando em frequência de até 240 MHz, 520 KB de memória de acesso aleatório (RAM, do inglês *Random Access Memory*) interna e conectividade sem fio WiFi 802.11 b/g/n e *Bluetooth* (ESPRESSIF SYSTEMS, 2026). O *chip* disponibiliza ainda pinos de entrada e saída de uso geral (GPIO, do inglês *General Purpose Input/Output*) e interfaces de comunicação com periféricos, como SPI, I2C e UART, além de conversores analógico-digitais. Essa combinação de conectividade nativa e preço acessível explica sua presença recorrente em trabalhos de ensino, como o de Duarte et al. (2024).

O Arduino Mega 2560 ocupa a outra ponta das placas usadas no ensino. É baseado no

microcontrolador ATmega2560, de 8 bits, operando a 16 MHz, com 8 KB de RAM, 54 pinos digitais, dos quais 15 geram sinais de modulação por largura de pulso (PWM, do inglês *Pulse Width Modulation*), e 16 entradas analógicas, sem conectividade sem fio (ARDUINO, 2026). Com menos memória, nenhuma conexão de rede e uma pinagem diferente, ele permite verificar se a plataforma se adapta a placas de recursos bem distintos a partir apenas do perfil de *hardware*.

Duas características comuns às duas placas importam diretamente para a proposta. Uma delas é a forma de programação, em que o código é escrito em C/C++ em uma IDE, como a Arduino IDE ou o PlatformIO, compilado no computador do estudante e gravado na placa por conexão serial. A outra é a restrição de recursos. A quantidade limitada de memória e a frequência de operação modesta impõem limites ao código, que precisa respeitar o consumo de memória e as particularidades de cada periférico. Essas duas características delimitam o que a plataforma proposta precisa produzir, ou seja, código-fonte compilável na IDE e coerente com os pinos e os periféricos efetivamente disponíveis na placa, informação fornecida ao modelo por meio do perfil de *hardware* descrito no Capítulo 3.

2.2 Modelos de linguagem de grande porte

Um modelo de linguagem de grande porte é uma rede neural treinada para prever o próximo elemento de uma sequência de texto a partir dos elementos anteriores. Esses modelos são construídos sobre a arquitetura *Transformer*, proposta por Vaswani et al. (2017), cujo mecanismo de atenção permite que cada elemento da sequência seja processado levando em conta o contexto completo em que aparece. Treinados sobre grandes volumes de texto, modelos com bilhões de parâmetros passam a executar tarefas para as quais não foram explicitamente treinados, a partir apenas da descrição da tarefa no texto de entrada.

A geração de código é uma dessas tarefas. Como o código-fonte de repositórios públicos compõe parte dos dados de treinamento, os modelos aprendem a associar descrições em linguagem natural a trechos de código correspondentes. Chen et al. (2021) avaliam modelos treinados com código e mostram que eles produzem funções corretas a partir de comentários descritivos, embora a taxa de acerto caia à medida que o problema se torna mais complexo e passa a depender de detalhes não expressos no enunciado.

Essa dependência torna o contexto fornecido ao modelo determinante para a qualidade do resultado. O texto enviado ao modelo, chamado de *prompt*, pode conter, além do pedido do usuário, informações que o modelo não teria como inferir. No caso de placas didáticas, essas informações são a identificação dos componentes conectados e dos pinos correspondentes; sem elas, o modelo tende a gerar código para uma configuração genérica, que não corresponde à placa em uso. Fruett et al. (2024) exploram essa propriedade ao fornecer ao ChatGPT uma descrição textual do *hardware* da BitDogLab antes de solicitar o código. A estratégia, conhecida como *prompting*, dispensa o ajuste fino do modelo, que exigiria dados rotulados e capacidade computacional muito superior.

Durante algum tempo, empregar um LLM implicava recorrer a serviços comerciais em

nuvem, pois apenas eles dispunham da infraestrutura necessária. A publicação de modelos abertos alterou esse quadro. O Llama 3 (Grattafiori et al., 2024), desenvolvido pela Meta, é distribuído em versões de diferentes tamanhos, e técnicas de quantização reduzem a precisão numérica dos parâmetros, diminuindo o espaço ocupado em memória e permitindo a execução em computadores comuns, com perda limitada de qualidade. Ferramentas como o Ollama (OLLAMA, 2026) encapsulam esse processo, pois obtêm o modelo, gerenciam sua execução e expõem uma API HTTP local, por meio da qual outras aplicações enviam *prompts* e recebem as respostas.

A escolha entre nuvem e execução local altera o compromisso entre as partes envolvidas. O processamento em nuvem oferece acesso a modelos maiores e dispensa investimento em infraestrutura, mas implica custo por uso, dependência de conexão com a internet e envio dos dados do usuário a terceiros. O processamento local mantém os dados na instituição, elimina o custo por requisição e permite o funcionamento em rede isolada, ao preço de exigir um servidor dedicado e de limitar o tamanho do modelo à capacidade desse servidor. Em um laboratório de ensino, no qual as requisições se concentram em poucos horários e o conteúdo é produzido por estudantes, a segunda configuração é a mais adequada, e é a adotada neste trabalho.

2.3 Aplicações *web* e APIs REST

Aplicações *web* seguem a arquitetura cliente-servidor, na qual as responsabilidades são divididas entre dois papéis. O cliente apresenta a interface e envia requisições, enquanto o servidor mantém os dados e a lógica de negócio e devolve as respostas. A separação permite que o cliente seja apenas o navegador do estudante, sem instalação de *software* adicional, e que o processamento fique concentrado em uma máquina da rede local, que pode atender vários clientes simultaneamente.

O estilo arquitetural REST, formulado por Fielding (2000), define um conjunto de restrições para sistemas distribuídos sobre a *web*. Entre elas estão a própria separação cliente-servidor, a ausência de estado no servidor, segundo a qual cada requisição carrega toda a informação necessária para ser atendida, e a interface uniforme, que identifica cada recurso por um identificador uniforme de recurso (URI, do inglês *Uniform Resource Identifier*) e o manipula por meio de representações padronizadas.

Na prática, uma API REST sobre o protocolo de transferência de hipertexto (HTTP, do inglês *Hypertext Transfer Protocol*) associa as operações aos métodos do protocolo (GET para leitura, POST para criação, PUT para atualização e DELETE para remoção) e sinaliza o desfecho de cada requisição por códigos de estado. As representações dos recursos trafegam usualmente na notação de objetos JavaScript (JSON, do inglês *JavaScript Object Notation*), formato textual leve, independente de linguagem e nativamente interpretado pelos navegadores.

Na plataforma proposta, esse estilo estrutura duas comunicações distintas. Na primeira, o navegador é cliente da API: envia a placa escolhida e a descrição da atividade e recebe o código gerado. Na segunda, a própria API é cliente do serviço que executa o modelo, ao qual envia

por HTTP o *prompt* montado com o perfil de *hardware* e do qual recebe a resposta. As duas comunicações ocorrem inteiramente dentro da rede local da instituição, o que dispensa acesso à internet durante o uso.

2.4 Trabalhos relacionados

Os trabalhos apresentados na seção 2.1 atacam o custo e a rigidez dos recursos didáticos, mas tratam a programação da placa como etapa já dominada pelo estudante. Duarte et al. (2024), Souza et al. (2014) e Souza, Silva e Fernandes (2016) entregam *hardware* de baixo custo e, no segundo caso, a capacidade de simular diferentes plantas, sem oferecer apoio à escrita do código. Parreira et al. (2021) removem a necessidade do dispositivo físico ao substituí-lo por simulação, o que viabiliza o ensino remoto, mas mantém a mesma barreira de programação e afasta o estudante do comportamento real do *hardware*.

O trabalho de Fruett et al. (2024) é o mais próximo desta proposta, por ser o único a tratar a barreira de programação diretamente. Nele, a descrição textual do *hardware* da BitDogLab é fornecida a um LLM, que gera e corrige os códigos da placa. A solução, porém, apoia-se em um serviço comercial em nuvem, o que impõe dependência de conexão com a internet, custo associado ao serviço e envio das interações dos estudantes a terceiros, além de exigir o carregamento manual da descrição do *hardware*.

O Quadro 1 resume essa comparação. A lacuna que este trabalho aborda está na última coluna, já que nenhum dos trabalhos analisados oferece apoio automatizado à programação com o modelo executado na infraestrutura da própria instituição.

Quadro 1 — Comparação entre os trabalhos relacionados e a proposta

Trabalho	**Recurso didático**	**Apoio à programação**	**Depende de nuvem**
Duarte et al. (2024)	Sistema embarcado de aquisição e controle em ESP32	Não	Não
Souza et al. (2014) e Souza, Silva e Fernandes (2016)	Simulação de plantas dinâmicas em tempo real	Não	Não
Parreira et al. (2021)	Laboratório virtual com *softwares* gratuitos	Não	Não
Fruett et al. (2024)	Placa aberta BitDogLab	Sim, via ChatGPT	Sim
Este trabalho	Placa didática programada pelo estudante	Sim, via LLM local	Não

Fonte: elaborado pelo autor.

3 METODOLOGIA

Este capítulo descreve os procedimentos adotados para alcançar os objetivos específicos definidos na seção 1.1. Cada seção corresponde a um desses objetivos, e a Figura 1 apresenta a visão geral da arquitetura proposta.

3.1 Requisitos e arquitetura

Os requisitos da plataforma foram derivados de duas fontes: o problema descrito no Capítulo 1, em que o estudante precisa obter código funcional para a placa sem dominar previamente a programação do microcontrolador, e as limitações identificadas na solução de Fruett et al. (2024), discutidas na seção 2.4. Essas limitações são a dependência de serviço comercial em nuvem, o envio das interações a terceiros e as etapas manuais de carregamento da descrição do *hardware*. Cada limitação foi convertida em um requisito da plataforma. O Quadro 2 apresenta o conjunto resultante.

Quadro 2 — Requisitos funcionais e não funcionais da plataforma

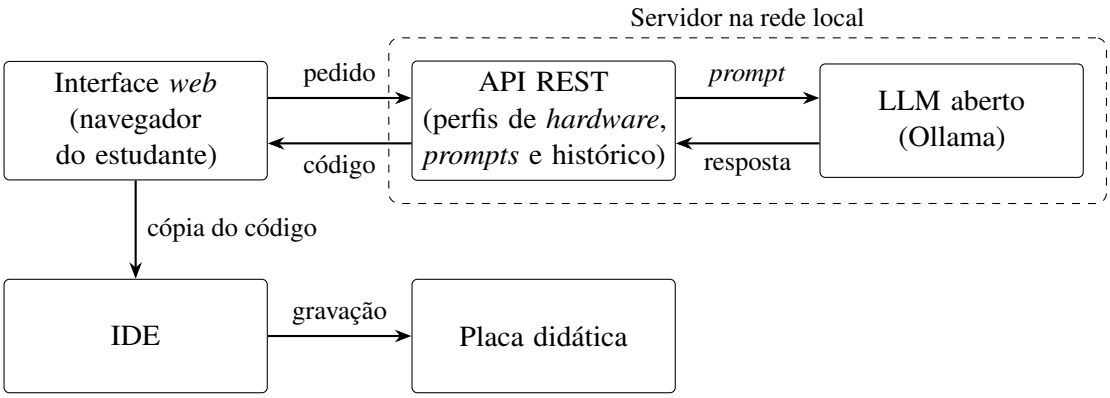
Identificador	**Descrição**
RF01	Permitir que o estudante selecione a placa alvo entre os perfis cadastrados
RF02	Permitir que o estudante descreva a atividade desejada em linguagem natural
RF03	Montar automaticamente o *prompt*, combinando o perfil de *hardware* da placa selecionada com a descrição da atividade
RF04	Submeter o *prompt* ao modelo e devolver o código gerado acompanhado de uma explicação
RF05	Manter o histórico da conversa, permitindo que o estudante refine o código a partir do resultado anterior
RF06	Indicar ao estudante quais componentes e pinos o código gerado utiliza e como ligá-los à placa
RNF01	Operar integralmente na rede local, sem depender de acesso à internet nem de serviços pagos
RNF02	Dispensar instalação de *software* no computador do estudante, além do navegador e da IDE
RNF03	Permitir o cadastro de novas placas por edição de arquivo de configuração, sem alteração do código-fonte

Fonte: elaborado pelo autor.

A arquitetura foi definida a partir desses requisitos e separa a plataforma em três partes, apresentadas na Figura 1. A interface *web* executa no navegador do estudante e concentra apenas a apresentação, o que atende ao RNF02. A API REST executa em um servidor da rede local e concentra as regras da plataforma. Ela guarda os perfis de *hardware*, monta os *prompts*, mantém o histórico das conversas e identifica os pinos usados pelo código gerado, atendendo aos requisitos RF01 a RF06. O modelo de linguagem executa no mesmo servidor, por meio do Ollama, e é acessado pela API por HTTP.

Duas consequências dessa separação orientaram o projeto. Primeiramente, a montagem do *prompt* fica inteiramente do lado do servidor, de modo que o estudante nunca manipula a descrição do *hardware*. Essa escolha elimina a etapa manual de carregamento da descrição apontada no trabalho de Fruett et al. (2024). A cópia do código para a IDE continua a cargo do estudante, pois a gravação na placa pela plataforma está fora do escopo deste trabalho. Além disso, o modelo é acessado por uma interface bem definida, o que permite trocá-lo sem alterar a interface *web* nem as regras da API, recurso utilizado no procedimento de avaliação descrito na seção 3.4.

Figura 1 — Arquitetura proposta para a plataforma



Fonte: elaborado pelo autor.

3.2 Implementação da API e integração com o LLM

A API foi implementada em Python com o *framework* FastAPI, executado pelo servidor *web* Uvicorn. A escolha considerou o suporte nativo do *framework* à validação de dados e à documentação automática dos recursos, e a existência de uma biblioteca oficial do Ollama para Python, que dispensa a construção manual das requisições ao modelo.

O código foi organizado em camadas, cada uma com uma responsabilidade única. As rotas tratam apenas do protocolo HTTP: recebem a requisição, validam o corpo e devolvem a resposta. Os serviços concentram as regras da plataforma, sendo responsáveis por carregar o perfil da placa, montar o *prompt*, registrar o histórico, acionar o modelo e identificar os pinos usados pelo código gerado. O acesso ao modelo e o acesso aos arquivos das conversas ficam isolados em módulos próprios, de modo que nenhuma outra parte do código depende do Ollama ou da forma de armazenamento. Os esquemas descrevem os formatos de entrada e saída, e um módulo de configuração centraliza os parâmetros ajustáveis, lidos de variáveis de ambiente, entre eles o nome do modelo utilizado.

Os perfis de *hardware* são arquivos JSON armazenados em um diretório do servidor, um por placa, carregados na inicialização. Cada perfil declara o identificador e o nome da placa, o microcontrolador, a linguagem e o *framework* de destino do código, a lista de componentes

disponíveis com os respectivos pinos e tipo, e observações sobre restrições de uso e bibliotecas. O perfil pode declarar ainda a disposição dos pinos na placa e a forma de ligar cada componente, informações usadas apenas pela interface. Cadastrar uma nova placa consiste em acrescentar um arquivo ao diretório e reiniciar o servidor, sem alterar o código-fonte, conforme o RNF03. A plataforma acompanha perfis para o ESP32 DevKit V1 e para as placas Arduino UNO R3, UNO R4, Mega 2560 e Nano, todos com o mesmo conjunto de componentes: o LED embutido, três LEDs externos, um botão, um *buzzer*, um sensor de temperatura e umidade DHT22, um potenciômetro, um *display* I2C e um módulo relé. O ESP32 e o Arduino Mega 2560 são as placas utilizadas na avaliação, e o Apêndice A apresenta os seus perfis.

A montagem do *prompt* é feita pela API a cada requisição e resulta em uma lista de mensagens, enviadas ao modelo nesta ordem: uma mensagem de sistema, que define o papel do modelo e o formato esperado da resposta (um único bloco com o código-fonte completo, seguido de uma explicação curta em português) e traz o perfil de *hardware* da placa selecionada, serializado em JSON; as mensagens anteriores da conversa, quando houver; e a descrição da atividade escrita pelo estudante. Do perfil, seguem para o modelo apenas os componentes, os pinos e as observações; a disposição dos pinos e as instruções de ligação, destinadas ao estudante, ficam de fora, o que reduz o tamanho do *prompt*. A instrução de sistema pede também que cada pino seja declarado como uma constante nomeada, o que facilita a identificação dos pinos descrita adiante. O histórico de cada conversa é gravado em arquivo JSON, o que permite o refinamento previsto no RF05 e preserva o registro entre reinicializações do servidor.

A comunicação com o modelo é feita pela biblioteca oficial do Ollama, que encapsula as chamadas HTTP ao serviço em execução no mesmo servidor. O nome do modelo é lido de uma variável de ambiente na inicialização do servidor, e não fixado no código, o que permite repetir a mesma atividade com modelos diferentes apenas reiniciando o servidor com outro valor, sem alteração no código. A resposta do modelo é então separada em duas partes: o primeiro bloco de código encontrado é tratado como o código gerado, e o texto restante, como a explicação.

Para atender ao RF06, a API identifica os pinos usados pelo código de cada resposta a partir da leitura do próprio código-fonte. São localizados os argumentos de pino em chamadas como `pinMode`, `digitalWrite`, `analogRead` e `tone` e na criação de objetos de bibliotecas, com a resolução das constantes nomeadas, e os pinos encontrados são comparados aos componentes do perfil. O resultado indica os componentes que o código utiliza, os que foram movidos para outro pino ao longo da conversa e os pinos usados sem componente ligado, situação que costuma revelar um pino inventado pelo modelo. Pinos calculados durante a execução do programa não são identificados.

Para a implantação, a API e o Ollama foram empacotados em contêineres Docker, iniciados em conjunto pelo Docker Compose. Na primeira inicialização, o modelo é baixado para um volume de armazenamento; a partir daí, a plataforma funciona sem acesso à internet, conforme o RNF01.

3.3 Desenvolvimento da interface *web*

A interface foi construída com HTML, CSS e JavaScript puros, sem *framework* de *front-end* e sem etapa de compilação. A decisão decorre do RNF02 e do escopo enxuto do trabalho. Dessa forma, os arquivos são servidos como conteúdo estático pela própria API, de modo que subir o servidor é a única operação necessária para colocar a plataforma em uso, e o estudante precisa apenas de um navegador. Nenhum recurso da interface é carregado da internet, e até a fonte tipográfica é distribuída com a plataforma, o que mantém a interface funcional em uma rede isolada.

O código do lado do cliente segue a mesma separação adotada na API. Os módulos de serviço concentram as chamadas à API, os componentes tratam da montagem dos elementos de tela e os utilitários reúnem as funções de formatação. A comunicação usa a interface *fetch* do navegador, com JSON nos dois sentidos, e todos os textos exibidos ao estudante, em português, ficam reunidos em um único módulo.

A interface é composta por uma única tela com a aparência de um terminal de linha de comando e tem um pato de borracha como personagem, em referência à técnica de depuração em que o programador explica o problema a um pato. Na barra superior, o estudante escolhe a placa em uma lista preenchida a partir dos perfis cadastrados. Na linha de comando, na parte inferior, descreve a atividade em texto livre. Na área central, recebe as respostas do modelo, e cada uma exibe o código como uma nova versão numerada, acompanhado de uma ação de cópia, da explicação gerada e do modelo e do tempo de geração. As mensagens anteriores permanecem visíveis acima da mais recente, como em um terminal, o que permite pedir ajustes sobre o código já produzido sem reescrever a descrição inicial. O pato indica o estado da plataforma: aguardando o pedido, gerando o código, concluído ou com erro.

Ao lado da conversa, um painel exibe o desenho da placa com a disposição dos pinos e indica qual placa selecionar na Arduino IDE, atendendo ao RF06. Antes de qualquer código, o painel destaca apenas os pinos que quase todo circuito utiliza, como os de alimentação e de terra, com a função de cada um. Para cada versão do código, mostra somente o que ela utiliza: os pinos ocupados, o desenho de cada componente com o pino ao qual cada terminal deve ser ligado, um aviso quando um componente mudou de pino em relação às versões anteriores e um alerta para pinos usados sem componente ligado. Algumas palavras digitadas na linha de comando são tratadas pela própria interface, sem consultar o modelo, como `help`, que lista os comandos disponíveis, `pinout`, que abre o painel da placa, e `clear`, que inicia uma nova conversa. O identificador da conversa fica no endereço da página, de modo que recarregá-la ou compartilhar o endereço reabre a mesma conversa.

3.4 Procedimento de avaliação

A avaliação foi conduzida em ambiente controlado, com toda a plataforma em execução local. O servidor de testes é um Mac mini de 2024 com processador Apple M4 e 16 GB de

memória unificada, que executa o Ollama e a API; o Ollama processa o modelo na unidade de processamento gráfico (GPU, do inglês *Graphics Processing Unit*) integrada ao processador. O mesmo computador executa o navegador e a IDE, com as placas conectadas por USB (do inglês *Universal Serial Bus*). A configuração é relevante porque o tempo de resposta do modelo depende diretamente dela.

Foram comparados dois modelos abertos de porte semelhante, distribuídos pelo Ollama com quantização de 4 bits (Q4_K_M): o Qwen2.5-Coder 7B, com 7,6 bilhões de parâmetros e treinado especificamente para código (Hui et al., 2024), e o Llama 3.1 8B, com 8,0 bilhões de parâmetros e de uso geral (Grattafiori et al., 2024). Os dois foram executados pelo mesmo servidor e acessados pela mesma configuração da plataforma, alterando-se apenas o nome do modelo; cada um ocupou cerca de 5 GB de memória e foi processado inteiramente na GPU. Antes da série de cada modelo, uma requisição de aquecimento, descartada, carregou o modelo em memória.

O conjunto de avaliação reúne dez atividades práticas de complexidade crescente, apresentadas no Quadro 3, divididas igualmente entre as duas placas. As que dependem de recursos próprios do ESP32 ficaram com ele: a conexão WiFi, nas atividades A8 a A10, a geração de PWM, que no ESP32 usa funções próprias, alteradas entre versões do seu núcleo, na A2, e o barramento I2C em pinos configuráveis, na A6. As outras cinco foram executadas no Arduino Mega 2560. As atividades foram escolhidas para cobrir os recursos mais usados em disciplinas introdutórias (saídas digitais, entradas analógicas, comunicação com periféricos e conectividade de rede) e são descritas à plataforma no mesmo nível de detalhe em que um roteiro de laboratório as apresentaria ao estudante.

Quadro 3 — Atividades práticas utilizadas na avaliação

Atividade	**Descrição**	**Complexidade**	**Placa**
A1	Piscar o LED embutido em intervalo fixo	Baixa	Mega 2560
A2	Variar a intensidade de um LED por PWM	Baixa	ESP32
A3	Ler o estado de um botão e acionar um LED	Baixa	Mega 2560
A4	Emitir uma sequência de tons em um *buzzer*	Média	Mega 2560
A5	Ler um sensor de temperatura e imprimir as leituras na porta serial	Média	Mega 2560
A6	Exibir as leituras de um sensor em um *display* I2C	Média	ESP32
A7	Acionar LEDs distintos conforme faixas de leitura de um potenciômetro	Média	Mega 2560
A8	Conectar a placa a uma rede WiFi e registrar o endereço obtido	Alta	ESP32
A9	Disponibilizar as leituras de um sensor em uma página servida pela placa	Alta	ESP32
A10	Acionar um atuador a partir de requisições HTTP recebidas pela placa	Alta	ESP32

Fonte: elaborado pelo autor.

Cada atividade foi submetida três vezes a cada modelo, em conversas independentes, para reduzir o efeito da variação entre gerações. Os pedidos, reproduzidos no Apêndice B, foram enviados pela API da plataforma, com as mesmas requisições que a interface faz, e o código de cada resposta foi salvo sem alterações como um projeto da IDE. Nas atividades que usam WiFi, o pedido indica as constantes que guardam o nome e a senha da rede, preenchidas apenas na hora da gravação, de modo que esses dados não passam pelo modelo. Na IDE ficaram instaladas apenas as bibliotecas previstas no perfil da placa. Cada código foi então compilado, gravado na placa da atividade e observado em execução. Considerou-se funcional o código que compila sem erros e produz o comportamento descrito no enunciado da atividade. Quando o resultado não atendeu a esse critério, o erro foi informado à plataforma pela própria conversa, em até duas rodadas de refinamento, e o desfecho foi registrado separadamente.

Duas métricas foram coletadas. A taxa de códigos funcionais foi calculada sobre o total de gerações e desmembrada em duas situações: código funcional na primeira geração e código funcional após refinamento. Já o tempo de resposta foi medido pela própria API entre o envio do *prompt* ao modelo e o recebimento da resposta completa, e reportado como média e desvio padrão das repetições. A plataforma configura o Ollama para manter o modelo carregado em memória por 30 minutos após a última requisição; assim, apenas a primeira geração após um período de inatividade inclui o tempo de carregamento do modelo, que a API registra separadamente. A medição isola o tempo do modelo, ou seja, não inclui a digitação da descrição pelo estudante nem a compilação e a gravação do código, que dependem da IDE e não da plataforma.

4 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados na mesma ordem das seções do Capítulo 3. A seção 4.1 descreve a plataforma implementada e o mecanismo que atende cada requisito. A seção 4.2 apresenta os recursos da API e um exemplo de uso. A seção 4.3 apresenta a interface. Por fim, a seção 4.4 reúne as medições obtidas nas dez atividades e discute os resultados frente aos trabalhos relacionados.

4.1 Requisitos e arquitetura

A plataforma, denominada Duckoder, foi implementada segundo a arquitetura apresentada na Figura 1, com as três partes em execução: a interface *web* no navegador do estudante, a API REST em um servidor da rede local e o modelo de linguagem sob o Ollama no mesmo servidor. Nenhuma das partes depende de acesso à internet durante a operação. O código-fonte está disponível em https://github.com/ctrl-vini/duckoder.

O Quadro 4 relaciona cada requisito definido no Quadro 2 ao mecanismo da implementação que o atende. A verificação foi feita exercitando cada requisito manualmente pela interface, no servidor descrito na seção 3.4, em uma conversa com o ESP32 composta por um pedido inicial e um pedido de refinamento, apresentada na seção 4.3. A lista de placas exibiu os cinco perfis cadastrados (RF01), e o pedido escrito em texto livre (RF02) foi respondido com código e explicação (RF03 e RF04). O código gerado usou os pinos que o perfil atribui ao LED vermelho e ao botão sem que o estudante os informasse, o que confirma que a descrição do *hardware* foi inserida no *prompt* pelo servidor. O pedido de refinamento foi aplicado sobre o código anterior (RF05), e o painel da placa apresentou os componentes usados por cada versão e a mudança de pino do LED (RF06). Todas as requisições registradas pelo navegador foram dirigidas ao próprio servidor da plataforma, e a única comunicação de saída da API é a chamada ao Ollama no mesmo servidor (RNF01); do lado do estudante, bastou o navegador (RNF02). Os perfis das quatro placas Arduino foram acrescentados à plataforma em uma alteração composta apenas por novos arquivos JSON, sem mudança no código-fonte (RNF03).

Além da verificação manual, o código conta com 87 testes automatizados, que substituem o modelo por uma implementação simulada e são executados a cada alteração enviada ao repositório. Todos os requisitos foram atendidos; a seção 4.3 discute uma limitação observada no painel da placa.

4.2 API REST

Os recursos expostos pela API são apresentados no Quadro 5. Os caminhos e os campos seguem a língua inglesa, como o restante do código-fonte; apenas os textos lidos pelo estudante estão em português. Os quatro primeiros recursos da lista atendem ao fluxo principal, do RF01 ao RF05; o perfil completo da placa e o recurso de uso de pinos alimentam o painel da placa

Quadro 4 — Mecanismo de implementação que atende cada requisito

Requisito	**Mecanismo**
RF01	Lista de placas montada na interface a partir dos perfis lidos do diretório de configuração
RF02	Linha de comando da interface, cujo texto é enviado à API sem tratamento prévio
RF03	Serviço de montagem do *prompt*, que monta a lista de mensagens enviada ao modelo: instrução de sistema com o perfil da placa, histórico e pedido
RF04	Recurso de envio de mensagem, que aciona o modelo e devolve código e explicação em JSON
RF05	Histórico gravado em arquivo JSON por conversa e reenviado ao modelo a cada nova mensagem
RF06	Serviço que identifica os pinos no código gerado e painel da placa com as ligações de cada versão do código
RNF01	Ollama e API no mesmo servidor da rede local; nenhuma chamada a serviço externo nem recurso da interface carregado da internet
RNF02	Interface servida como conteúdo estático pela própria API, sem etapa de compilação
RNF03	Perfis em arquivos JSON independentes, validados e carregados na inicialização

Fonte: elaborado pelo autor.

(RF06); e o último informa a versão da plataforma em execução.

Quadro 5 — Recursos expostos pela API

Método	**Caminho**	**Descrição**
GET	/boards	Lista as placas cadastradas
POST	/conversations	Cria uma conversa para a placa escolhida
POST	/conversations/{id}/messages	Envia a descrição da atividade e devolve a resposta do modelo
GET	/conversations/{id}	Recupera o histórico de uma conversa
GET	/boards/{id}	Devolve o perfil completo de uma placa, com a disposição dos pinos e as ligações
GET	/conversations/{id}/messages/{message_id}/pin-usage	Indica os componentes e os pinos usados pelo código de uma resposta
GET	/version	Informa o nome e a versão da plataforma

Fonte: elaborado pelo autor.

O Quadro 6 mostra uma requisição de geração de código e a resposta correspondente, retiradas da conversa apresentada na seção 4.3. O estudante envia apenas a descrição da atividade, no campo `content`, pois a placa já está associada à conversa e o perfil de *hardware* é inserido no *prompt* pelo servidor, sem passar pelo cliente.

O campo `content` guarda a resposta do modelo na íntegra, e as duas partes separadas pela API ficam nos campos `code` e `explanation`. O objeto `metrics` registra o modelo que gerou a resposta, o intervalo entre o envio do *prompt* ao modelo e o recebimento da resposta completa (`elapsed_ms`), a parcela desse intervalo gasta no carregamento do modelo em memória

Quadro 6 — Exemplo de requisição e resposta da API

```
POST /conversations/1910f627.../messages
{
  "content": "Acenda o LED vermelho enquanto o botão estiver pressionado"
}

201 Created
{
  "id": "e30b4912...",
  "role": "assistant",
  "content": "<resposta completa do modelo>",
  "code": "const int BUTTON_PIN = 4;\nconst int RED_LED_PIN = 27;\n...",
  "explanation": "Este código acende o LED vermelho enquanto ...",
  "metrics": {
    "model": "qwen2.5-coder:7b",
    "elapsed_ms": 24113,
    "load_ms": 13617,
    "output_tokens": 129,
    "prompt_version": "v6"
  },
  "created_at": "2026-10-13T09:16:41.321725Z"
}
```

Fonte: elaborado pelo autor.

(`load_ms`), a quantidade de *tokens* gerados e a versão da instrução de sistema utilizada. São esses campos que alimentam as medições de tempo apresentadas na seção 4.4, o que dispensa instrumentação externa durante os testes.

4.3 Interface *web*

A Figura 2 mostra a interface ao fim da conversa usada na verificação dos requisitos. A placa escolhida, o ESP32 DevKit V1, aparece na barra superior. O estudante pediu, em texto livre, que o LED vermelho acendesse enquanto o botão estivesse pressionado, e a plataforma devolveu a versão 1 do código, seguida da explicação gerada pelo modelo. Em seguida, o estudante informou que o GPIO 27 havia queimado e pediu que o LED fosse ligado em outro pino livre; a versão 2 manteve o restante do código e alterou apenas a constante do pino do LED. Cada versão traz as ações de cópia e de visualização das ligações, e cada resposta termina com o modelo que a gerou e o tempo de geração. A primeira resposta levou 24,1 s, dos quais 13,6 s foram gastos no carregamento do modelo em memória, por ser a primeira requisição após a inicialização do servidor; a segunda, com o modelo já carregado, levou 7,1 s.

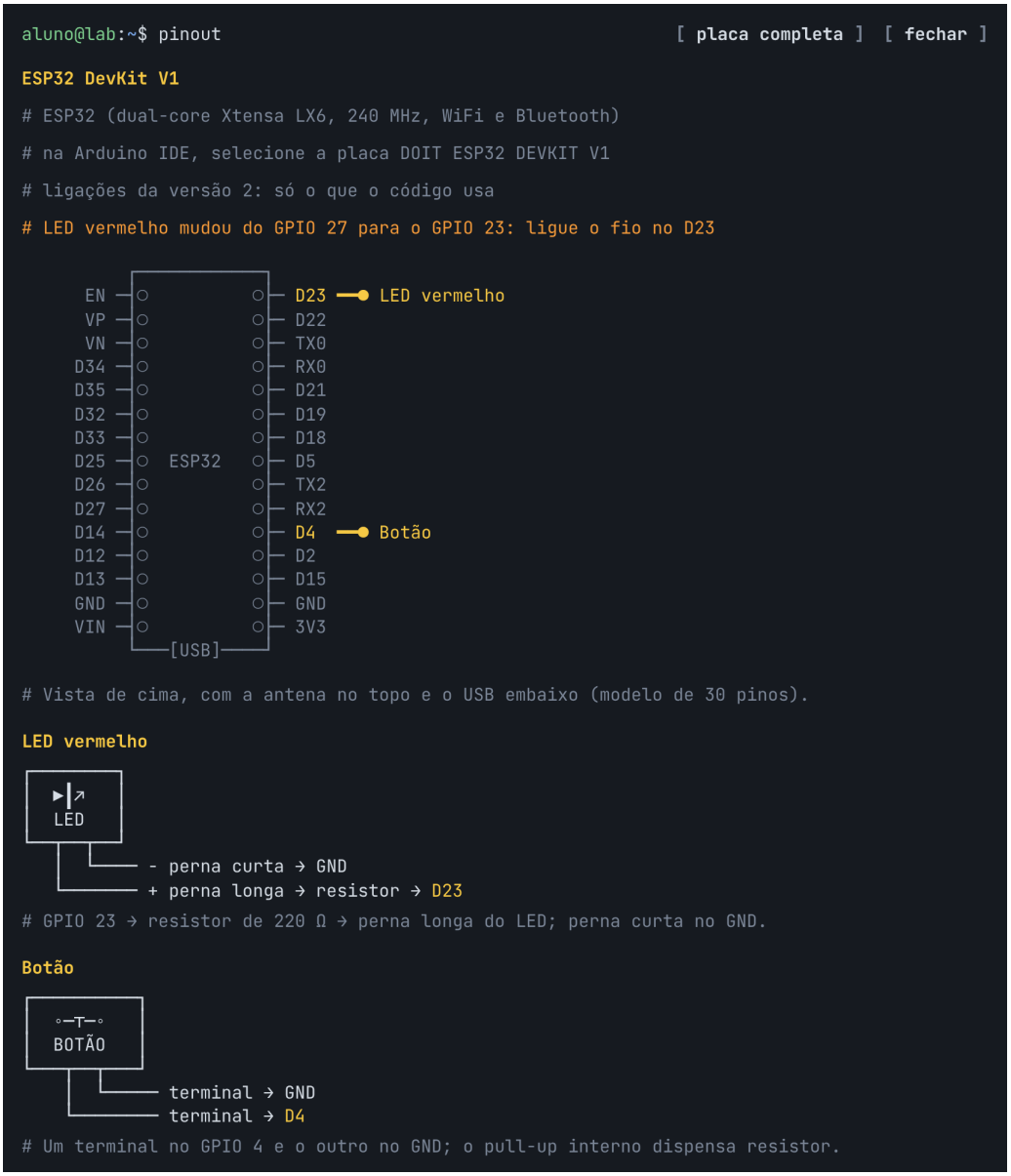
A Figura 3 mostra o painel da placa aberto na versão 2. O desenho destaca apenas os pinos usados pelo código, D23 e D4, com o nome do componente ligado a cada um, e o painel informa qual placa selecionar na Arduino IDE. Como o LED vermelho estava no GPIO 27 na versão anterior, um aviso indica que o componente mudou de pino e onde ligar o fio. Abaixo do desenho, cada componente aparece com o pino ao qual cada terminal deve ser ligado.

Figura 2 — Interface do Duckoder ao fim de uma conversa com refinamento



Fonte: elaborado pelo autor.

Figura 3 — Painel da placa com as ligações da versão 2 do código



Fonte: elaborado pelo autor.

O exemplo também expõe um erro do modelo. As observações do perfil indicam os pinos livres a usar quando o estudante pede a troca de pino (Apêndice A), mas o modelo escolheu o GPIO 23, que o perfil atribui ao relé. O painel identificou corretamente a nova ligação do LED, mas não alerta quando o novo pino já pertence a outro componente do perfil. Assim, um estudante que mantivesse o relé ligado à placa teria os dois componentes no mesmo pino. Essa verificação fica como limitação da implementação atual.

4.4 Avaliação da plataforma

As dez atividades do Quadro 3 foram submetidas três vezes a cada modelo, conforme o procedimento da seção 3.4, totalizando 60 gerações de código: 30 por modelo, das quais 15 no ESP32 e 15 no Arduino Mega 2560. Todas as respostas trouxeram um bloco de código.

4.4.1 Taxa de códigos funcionais

A Tabela 1 apresenta, para cada atividade, a placa em que foi executada, quantas das três gerações produziram código funcional já na primeira resposta e quantas passaram a funcionar após as rodadas de refinamento.

Tabela 1 — Códigos funcionais por atividade, em três gerações por modelo

		Qwen2.5-Coder 7B		Llama 3.1 8B	
Atividade	Placa	1.ª geração	Após refino	1.ª geração	Após refino
A1	Mega 2560	3	0	3	0
A2	ESP32	3	0	3	0
A3	Mega 2560	3	0	3	0
A4	Mega 2560	3	0	3	0
A5	Mega 2560	3	0	1	0
A6	ESP32	2	1	0	0
A7	Mega 2560	1	0	3	0
A8	ESP32	3	0	2	1
A9	ESP32	1	0	2	1
A10	ESP32	3	0	0	1
Total		25	1	20	3

Fonte: elaborado pelo autor.

No Arduino Mega 2560, as 12 gerações do Qwen2.5-Coder nas atividades A1, A3, A4 e A5 funcionaram já na primeira resposta. Os códigos usaram os pinos que o perfil atribui a cada componente nessa placa, diferentes dos usados no ESP32, e a velocidade de 9600 baud que as observações do perfil indicam para o monitor serial. Em uma das gerações da A5, a primeira compilação falhou porque a biblioteca do DHT22 ainda não estava instalada na IDE, em desacordo com o procedimento da seção 3.4. Ao receber a mensagem de erro, o modelo

não gerou código novo e indicou a instalação da biblioteca; instalada a biblioteca, o código original funcionou. Por se tratar de falha na preparação do ambiente, e não no código gerado, essa geração foi contada como funcional na primeira resposta.

Na A7, apenas uma das três gerações do Qwen2.5-Coder funcionou na primeira resposta. As outras duas compilaram, mas não dividiram corretamente o curso do potenciômetro. Em uma delas, a faixa foi calculada com a função `map`, convertendo a leitura de 0 a 1023 para um valor de 0 a 2; como o cálculo é feito com números inteiros, o resultado só chega a 2 no valor máximo da leitura. Na prática, o LED verde acendia na primeira metade do curso, o amarelo na segunda e o vermelho apenas no fim exato, o que resultou em duas faixas em vez de três. Na outra, a faixa foi obtida dividindo a leitura por 333; para leituras de 999 a 1023, o resultado é 3, valor que o código não associa a nenhum LED, e os três se apagavam no fim do curso. Os dois erros estão no tratamento dos limites da escala de 10 bits e foram classificados como comportamento diferente do pedido.

Nenhuma das duas foi recuperada nas duas rodadas de refinamento previstas. Na geração que dividia a leitura por 333, o modelo manteve a mesma divisão em todas as respostas seguintes e, em uma delas, moveu a leitura do potenciômetro para a função `setup`, de modo que o código deixou de responder ao potenciômetro; mesmo com mais mensagens do que as duas rodadas previstas, os LEDs continuaram apagando no fim do curso. Na geração que usava a função `map`, a primeira rodada recebeu por engano a descrição do defeito da outra geração. Nas rodadas seguintes, já com a descrição correta, o modelo manteve a função `map` e alterou apenas o acionamento dos LEDs, chegando a afirmar na explicação que as faixas estavam divididas igualmente. Uma divisão correta só apareceu na sexta mensagem, depois que o próprio pedido apontou a função `map` como causa do problema, o que exige do estudante justamente o diagnóstico que se esperava da plataforma. Os dois casos indicam a dificuldade do modelo em corrigir um erro de lógica a partir apenas da descrição do comportamento observado.

No Llama 3.1, 13 das 15 gerações no Mega funcionaram na primeira resposta, o mesmo total do Qwen2.5-Coder, mas as falhas ficaram em outra atividade. As gerações da A1, da A3 e da A4 funcionaram todas, igualmente com os pinos do perfil. Na A7, as três gerações funcionaram. Em vez de calcular a faixa com a função `map` ou com uma divisão, o modelo comparou a leitura com dois limites fixos, 340 e 680, o que cobre toda a escala de 0 a 1023 e evita o problema dos limites observado no Qwen2.5-Coder.

As duas falhas do Llama 3.1 ocorreram na A5. Em duas gerações, o objeto do sensor foi criado sem a chamada a `dht.begin()`, que inicializa o sensor, e o monitor serial exibiu `nan` no lugar da temperatura. Nenhuma delas foi recuperada nas duas rodadas previstas. Em uma, as respostas seguintes recorreram a métodos que não existem na biblioteca, como `available` e `isValidRead`, e declararam o sensor dentro da função `setup`, fora do alcance da função `loop`; mesmo com mais de dez mensagens de correção, incluindo pedidos para voltar à primeira versão, o modelo não chegou a um código que compilasse. Na outra, a primeira rodada usou `dht.begin()` como condição de um `if`, embora a função não retorne valor, e a segunda removeu

do construtor o tipo do sensor, de modo que as duas respostas deixaram de compilar; o código correto só veio na terceira rodada, fora do limite, e a geração foi contada como falha. A geração restante inicializou o sensor corretamente, mas acrescentou, sem que o pedido o mencionasse, a exibição da temperatura no *display* OLED descrito no perfil. A compilação falhou porque a biblioteca do *display*, prevista no perfil, não estava instalada na IDE, o mesmo problema de preparação do ambiente relatado para o Qwen2.5-Coder; instalada a biblioteca, o código original mostrou a temperatura no monitor serial, como pedido, e foi contado como funcional na primeira resposta.

No Mega, portanto, cada modelo produziu 13 códigos funcionais em 15 gerações, nenhum deles obtido por refinamento. As falhas, porém, não coincidiram. O Qwen2.5-Coder errou no tratamento dos limites da escala do potenciômetro, que o Llama 3.1 resolveu nas três gerações, enquanto o Llama 3.1 errou na inicialização do sensor, que o Qwen2.5-Coder fez corretamente em todas. Nos dois modelos, as respostas de correção tenderam a manter o erro original ou a introduzir erros novos, como métodos inexistentes na biblioteca, em vez de corrigir o defeito descrito.

No ESP32, as seis gerações da A2 funcionaram na primeira resposta nos dois modelos, todas com a função `analogWrite`, que as observações do perfil indicam no lugar das funções de PWM que deixaram de existir nas versões recentes do núcleo da placa. Na A8, as três gerações do Qwen2.5-Coder funcionaram na primeira resposta. Em uma delas, o monitor serial exibiu caracteres ilegíveis por estar configurado com velocidade diferente dos 115200 baud usados pelo código; ajustado o monitor, o endereço IP apareceu, e, como na A5, a geração foi contada como funcional na primeira resposta. No Llama 3.1, duas gerações da A8 funcionaram na primeira resposta, e a terceira omitiu a inclusão da biblioteca `WiFi.h`, erro de compilação corrigido na primeira rodada de refinamento.

Na A6, duas gerações do Qwen2.5-Coder funcionaram na primeira resposta, e a terceira não compilou por usar o comando `continue` fora de um laço, na função `loop`; a primeira rodada substituiu o comando por uma estrutura condicional, e o código passou a funcionar. As três gerações do Llama 3.1 falharam pelo mesmo motivo. Em todas elas, o objeto do *display* foi criado passando os pinos SDA e SCL, 21 e 22, que o perfil indica para a função `Wire.begin`, nos parâmetros que definem a largura e a altura da tela. O código compilou, mas a biblioteca passou a tratar o *display* como uma tela de 21 por 22 pixels, e apenas um fragmento do texto aparecia no canto superior. Nas duas rodadas de cada geração, o modelo alterou a posição do cursor, o tamanho da letra e a conversão dos valores, mas não o construtor do *display*, e nenhuma das três foi recuperada.

Na A9, o Qwen2.5-Coder produziu código funcional em apenas uma das três gerações, a que usou a biblioteca `WebServer`, do próprio núcleo da placa. As outras duas recorreram à biblioteca `ESPAsyncWebServer`, que não faz parte do perfil, e não compilaram. Em uma delas, o código chamava a função `handleClient`, que pertence à biblioteca `WebServer`; a primeira rodada manteve a função, atribuindo o erro à versão da biblioteca, e a segunda a trocou por uma

função `run`, também inexistente. Na outra, as constantes com o nome e a senha da rede foram declaradas como números inteiros; a primeira rodada corrigiu as constantes, mas passou a usar um pino sem declará-lo, e a segunda manteve a dependência da biblioteca externa, sem chegar a um código que compilasse. No Llama 3.1, duas gerações da A9 funcionaram na primeira resposta, e a terceira, que usava o objeto do servidor sem declará-lo, foi corrigida na primeira rodada.

Na A10, as três gerações do Qwen2.5-Coder funcionaram na primeira resposta, todas com a biblioteca `WebServer`; uma delas, porém, não inicializava a porta serial nem mostrava o endereço da placa, que o estudante precisa conhecer para acessar os endereços de controle. As três gerações do Llama 3.1 não compilaram pelo mesmo motivo, pois, em vez de criar um servidor, o código tentava ler as requisições por uma função `client` do objeto `WiFi`, que não existe. Apenas uma foi recuperada, na segunda rodada. Nas outras duas, as respostas seguintes recorreram a outras funções inexistentes do mesmo objeto, a uma biblioteca do ESP8266, placa diferente da selecionada, e a uma biblioteca que não existe; em uma delas, o código correto só veio na terceira rodada, fora do limite, e a geração foi contada como falha.

Somadas as duas placas, o Qwen2.5-Coder produziu código funcional na primeira resposta em 25 das 30 gerações (83%) e chegou a 26 (87%) com o refinamento; o Llama 3.1, em 20 (67%) e 23 (77%), respectivamente. O modelo treinado para código teve, portanto, o melhor desempenho, sobretudo na primeira resposta. A taxa caiu com a complexidade das atividades, mas de forma diferente nos dois modelos. Nas atividades de complexidade baixa, as 18 gerações funcionaram na primeira resposta. Nas de complexidade média, o Qwen2.5-Coder acertou 9 das 12 na primeira resposta, e o Llama 3.1, 7; nas de complexidade alta, 7 e 4 das 9, respectivamente, embora o refinamento tenha levado o Llama 3.1 ao mesmo total de 7. As falhas concentraram-se em poucas atividades e tenderam a se repetir nas três gerações do mesmo modelo, como no construtor do *display* na A6 e na leitura das requisições na A10, no Llama 3.1, o que indica lacunas do modelo sobre bibliotecas específicas, e não variação aleatória entre gerações.

Das 15 gerações que falharam na primeira resposta, 8 não compilaram e 7 compilaram, mas não produziram o comportamento pedido. O refinamento recuperou 4 delas, todas com erro de compilação, e nenhuma das 7 com comportamento diferente. A diferença está na informação que chega ao modelo. A mensagem do compilador aponta a linha e a causa do erro, enquanto o comportamento observado precisa ser descrito pelo estudante, e, nesses casos, as respostas de correção tenderam a alterar partes do código sem relação com o defeito. Mesmo entre os erros de compilação, metade não foi recuperada, em geral porque o modelo substituiu uma função inexistente por outra, também inexistente.

4.4.2 Tempo de resposta

A Tabela 2 apresenta o tempo de resposta do modelo por atividade, em segundos, como média e desvio padrão das três gerações. Os valores são os registrados pela API no campo `elapsed_ms`.

Tabela 2 — Tempo de resposta por atividade, em segundos

Atividade	Placa	Qwen2.5-Coder 7B	Llama 3.1 8B
A1	Mega 2560	5,8 ± 2,2	6,7 ± 1,2
A2	ESP32	8,5 ± 0,8	9,9 ± 0,5
A3	Mega 2560	8,3 ± 2,7	7,8 ± 0,2
A4	Mega 2560	13,0 ± 1,7	14,0 ± 3,3
A5	Mega 2560	11,3 ± 1,4	8,5 ± 3,8
A6	ESP32	21,9 ± 2,0	14,8 ± 1,2
A7	Mega 2560	12,3 ± 2,4	16,1 ± 1,4
A8	ESP32	9,4 ± 0,6	9,2 ± 1,1
A9	ESP32	24,2 ± 3,6	20,9 ± 5,2
A10	ESP32	23,9 ± 2,8	14,7 ± 1,3
Média		13,9 ± 6,9	12,3 ± 4,8

Fonte: elaborado pelo autor.

Nas 30 gerações do Qwen2.5-Coder, o tempo médio de resposta foi de 13,9 s, entre 3,4 e 27,8 s. O tempo acompanhou de forma praticamente linear a quantidade de *tokens* gerados, a cerca de 21 *tokens* por segundo, e por isso cresceu com o tamanho do código. As atividades mais demoradas foram a A6, a A9 e a A10, executadas no ESP32, que combinam sensor, *display* ou servidor *web* e resultam nos códigos mais extensos; as mais rápidas foram as de complexidade baixa no Mega, como a A1, com menos de 6 s em média.

No Llama 3.1, o tempo médio foi de 12,3 s, entre 5,6 e 26,5 s, a cerca de 19,5 *tokens* por segundo. A geração foi um pouco mais lenta por *token* que a do Qwen2.5-Coder, mas as respostas foram mais curtas, com 239 *tokens* em média contra 291, e o tempo médio ficou 1,6 s abaixo. A diferença concentrou-se no ESP32, sobretudo nas atividades A6 e A10, em que o Llama produziu códigos mais curtos; no Mega, as médias dos dois modelos foram próximas, de 10,2 s no Qwen2.5-Coder e 10,6 s no Llama. Em ambos os modelos, o tempo cresceu com a complexidade das atividades, mas a diferença entre eles ficou em poucos segundos, o que não altera a experiência de uso.

A requisição de aquecimento que antecedeu a série de cada modelo deixou o modelo carregado em memória, de modo que o carregamento não pesou nas gerações medidas, nas quais o tempo de carga registrado não passou de poucos milissegundos. O custo aparece na primeira requisição após a inicialização do servidor ou após 30 minutos sem uso. Na conversa da Figura 2, por exemplo, 13,6 s dos 24,1 s da primeira resposta foram de carregamento. Com respostas entre 3 e 28 s, o tempo observado é compatível com o uso durante uma aula quando poucos estudantes fazem pedidos ao mesmo tempo. Nessa configuração, o Ollama processa uma requisição por vez, e pedidos simultâneos entram em fila, de modo que a espera de cada estudante cresce com o número de pedidos à sua frente.

4.4.3 Discussão

Os resultados permitem responder à pergunta central deste trabalho, que é saber se um modelo aberto executado na infraestrutura da instituição é suficiente para a tarefa que Fruett et al. (2024) resolvem com um serviço comercial em nuvem. A estratégia de contexto é a mesma nos dois casos (a descrição do *hardware* precede o pedido do estudante), e a diferença está no modelo e em onde ele executa. Fruett et al. (2024) não medem a taxa de códigos funcionais, pois a avaliação da BitDogLab é feita pela percepção dos participantes de uma oficina, dos quais 64% destacaram a facilidade de programar com o auxílio da IA. Não há, portanto, um número a comparar diretamente, e este trabalho não mediu um modelo comercial nas mesmas atividades. Os resultados indicam, ainda assim, que um modelo local atende às atividades de complexidade baixa e média com poucas falhas, com o Qwen2.5-Coder em 83% de códigos funcionais na primeira resposta, e que as perdas se concentram em bibliotecas específicas, como a do *display* e as de servidor *web*, ponto em que modelos maiores, executados em servidores com mais memória, podem ser avaliados em trabalhos futuros. O ciclo de correção também é o mesmo nos dois trabalhos, em que o estudante devolve ao modelo o erro observado, e os resultados mostram que ele funciona melhor com as mensagens do compilador do que com a descrição de um comportamento.

Frente aos demais trabalhos comparados no Quadro 1, a contribuição é de outra natureza. Duarte et al. (2024) e Souza et al. (2014) reduzem o custo do recurso didático, e Parreira et al. (2021) remove a necessidade do dispositivo físico; nenhum deles atua sobre a etapa de escrita do código, que é justamente onde a plataforma proposta intervém. Os dois conjuntos de resultados são, portanto, complementares, já que a plataforma pode ser aplicada sobre qualquer um desses recursos didáticos, desde que exista um perfil de *hardware* correspondente.

Do ponto de vista de redes, a operação integralmente local mostrou-se estável, e todas as gerações e as requisições da verificação foram atendidas sem nenhuma falha de comunicação entre navegador, API e Ollama. A comunicação também não representou gargalo. A diferença entre o tempo medido pelo navegador e o tempo de inferência registrado pela API, que reúne a troca HTTP, a separação da resposta e a gravação da conversa, ficou entre 4 e 10 ms, e cada resposta ocupou no máximo 4,4 kB, frente a segundos de inferência. Como o navegador executou no próprio servidor, essa medida não inclui o meio físico da rede; ainda assim, mensagens desse tamanho acrescentariam tempo desprezível em uma rede local comum. Esse é o compromisso central discutido na seção 2.2, em que se troca o acesso a modelos maiores pela independência de conexão externa, pela ausência de custo por requisição e pela permanência dos dados na instituição.

Três limitações devem ser consideradas na leitura destes resultados. Em primeiro lugar, a amostra de dez atividades com três gerações cada oferece indicação de tendência, e não significância estatística; e, como cada atividade foi executada em uma única placa, os resultados não permitem comparar as duas placas na mesma atividade. Em segundo lugar, a avaliação do

comportamento do código foi feita pelo autor, e não por estudantes, de modo que a clareza das explicações geradas não foi medida junto ao público-alvo. Por fim, os tempos dependem do servidor utilizado, descrito na seção 3.4, e não se transferem diretamente para outras configurações.

5 CONCLUSÃO

Este trabalho partiu de uma barreira comum nas disciplinas práticas. Mesmo com placas didáticas acessíveis, programá-las ainda é difícil para estudantes iniciantes. A solução de Fruett et al. (2024) enfrenta essa barreira com um LLM comercial em nuvem, o que traz dependência de acesso à internet, custo por uso, envio das interações a terceiros e o carregamento manual da descrição do *hardware*. Para oferecer essa assistência sem essas restrições, foi desenvolvido o Duckoder, uma plataforma *web* integrada a um LLM aberto executado em um servidor da rede local, o que atende ao objetivo geral definido na seção 1.1.

Os quatro objetivos específicos foram cumpridos. Os requisitos e a arquitetura separaram a plataforma em interface *web*, API REST e modelo de linguagem, e a descrição de cada placa foi definida como um perfil em JSON, com os componentes, os pinos e as observações de uso. A API, implementada com o FastAPI, guarda esses perfis, monta o *prompt* a cada requisição, mantém o histórico das conversas e acessa o modelo por meio do Ollama, de modo que o estudante nunca manipula a descrição do *hardware*. A interface permite escolher a placa, descrever a atividade em linguagem natural, refinar o código na mesma conversa e consultar como ligar os componentes usados por cada versão do código. A verificação dos requisitos confirmou o funcionamento integralmente local, sem chamadas a serviços externos, e a inclusão de novas placas apenas com novos arquivos de perfil. Por fim, a plataforma foi avaliada em dez atividades práticas, divididas entre o ESP32 e o Arduino Mega 2560, com dois modelos abertos de porte semelhante.

Os resultados indicam que um modelo local é suficiente para as atividades típicas de disciplinas introdutórias. O Qwen2.5-Coder 7B, treinado para código, produziu código funcional na primeira resposta em 83% das gerações e em 87% após o refinamento; o Llama 3.1 8B, de uso geral, em 67% e 77%, respectivamente. Todas as gerações das atividades de complexidade baixa funcionaram na primeira resposta, e as falhas concentraram-se em poucas atividades, ligadas a bibliotecas específicas, como a do *display* e as de servidor *web*. O refinamento pela própria conversa mostrou-se eficaz para erros de compilação, cuja mensagem aponta a causa ao modelo, mas não recuperou nenhum caso em que o código compilava e se comportava de forma diferente da pedida. O tempo médio de resposta ficou entre 12 e 14 s, com respostas entre 3 e 28 s, e a comunicação entre navegador, API e modelo acrescentou apenas milissegundos, sem nenhuma falha durante a avaliação.

A principal contribuição do trabalho é mostrar que a estratégia de contexto adotada por Fruett et al. (2024), que consiste em apresentar o *hardware* ao modelo antes do pedido do estudante, pode ser automatizada e executada inteiramente na infraestrutura da instituição, com um modelo aberto e um computador de mesa com 16 GB de memória. Troca-se o acesso a modelos maiores pela independência de conexão externa, pela ausência de custo por requisição e pela permanência dos dados na instituição. Do ponto de vista de redes, a plataforma exemplifica um serviço de inteligência artificial oferecido na rede local, em que o tráfego entre os estudantes e o servidor é pequeno diante do tempo de processamento do modelo.

Esses resultados devem ser lidos com as limitações discutidas na subseção 4.4.3: a amostra de dez atividades com três gerações cada, a execução de cada atividade em uma única placa, a avaliação do comportamento feita pelo autor, e não por estudantes, e a dependência dos tempos em relação ao servidor utilizado. A plataforma também não verifica conflitos de pinos quando o modelo escolhe um pino já atribuído a outro componente, como mostrado na seção 4.3.

Como trabalhos futuros, sugerem-se:

a) integrar à plataforma a compilação e a gravação do código, com o reconhecimento da placa conectada por USB e sua seleção automática, de modo que as mensagens do compilador sejam devolvidas ao modelo sem intervenção do estudante, já que foram esses os erros que o refinamento conseguiu corrigir;

b) permitir a simulação do código e do circuito antes da gravação, para que o estudante compare o comportamento obtido com o esperado e descreva com mais precisão os erros de lógica, que o refinamento não recuperou nesta avaliação;

c) ampliar os perfis de *hardware*, com novos componentes, a criação de componentes personalizados pelo professor e a edição das ligações sugeridas pela plataforma, incluindo a verificação de conflitos de pinos;

d) avaliar modelos locais maiores ou especializados e compará-los, nas mesmas atividades, com serviços comerciais acessados por API, para medir quanto a execução local custa em qualidade dos códigos;

e) avaliar a plataforma com turmas de estudantes, medindo a clareza das explicações geradas e o efeito sobre a aprendizagem.

REFERÊNCIAS

ARDUINO. **Mega 2560 Rev3**. [S. l.], 2026. Disponível em: https://docs.arduino.cc/hardware/mega-2560/. Acesso em: 14 out. 2026.

CHEN, M.; TWOREK, J.; JUN, H.; YUAN, Q. et al. Evaluating large language models trained on code. **arXiv**, 2021. ArXiv: 2107.03374.

DUARTE, G. M.; JUNIOR, K. R. S.; OLIVEIRA, L. S.; LEITE, V. J. S. Sistema embarcado para aplicações em disciplinas experimentais. In: CONGRESSO BRASILEIRO DE AUTOMÁTICA, 25., 2024, Rio de Janeiro. **Anais** [...]. Rio de Janeiro: SBA, 2024. DOI: 10.20906/CBA2024/4231.

ESPRESSIF SYSTEMS. **ESP32 series datasheet**. Version 5.3. [S.l.], 2026. Disponível em: https://documentation.espressif.com/esp32_datasheet_en.html. Acesso em: 19 set. 2026.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. Tese (Doutorado em Information and Computer Science) — University of California, Irvine, 2000.

FRUETT, F.; BARBOSA, F. P.; FRAGA, S. C. Z.; GUIMARÃES, P. I. A. Empowering STEAM activities with artificial intelligence and open hardware: the BitDogLab. **IEEE Transactions on Education**, v. 67, n. 3, p. 462–471, jun. 2024. DOI: 10.1109/TE.2024.3377555.

GRATTAFIORI, A.; DUBEY, A.; JAUHRI, A. et al. The Llama 3 herd of models. **arXiv**, 2024. ArXiv: 2407.21783.

HUI, B.; YANG, J.; CUI, Z. et al. Qwen2.5-Coder technical report. **arXiv**, 2024. ArXiv: 2409.12186.

OLLAMA. **Ollama**. [S. l.], 2026. Disponível em: https://ollama.com. Acesso em: 19 set. 2026.

PARREIRA, P. F. P.; SILVA, T. R.; NOVAIS, H. A. de; LUZ, E. J. d. S.; Rêgo Segundo, A. K. Laboratório virtual de sistemas embarcados: uma abordagem de ensino remoto usando softwares gratuitos. In: SIMPÓSIO BRASILEIRO DE AUTOMAÇÃO INTELIGENTE, 15., 2021, On-line. **Anais** [...]. [S.l.]: SBA, 2021. p. 1405–1412. DOI: 10.20906/sbai2021/216514.

SOUZA, I. D. T. de; SILVA, S. N.; FERNANDES, M. A. C. Plataforma para simulação em tempo real de sistemas dinâmicos e *hardware-in-the-loop* em sistemas embarcados do tipo PSOC. **Proceeding Series of the Brazilian Society of Computational and Applied Mathematics**, v. 4, n. 1, 2016.

SOUZA, I. D. T. de; SILVA, S. N.; TELES, R.; FERNANDES, M. A. C. Plataforma de simulação em tempo real para sistemas dinâmicos. In: CONGRESSO BRASILEIRO DE AUTOMÁTICA, 20., 2014, Belo Horizonte. **Anais** [...]. Belo Horizonte: SBA, 2014. p. 867–874.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. **Advances in Neural Information Processing Systems**, v. 30, p. 5998–6008, 2017.

APÊNDICE A — DESCRIÇÃO DO *HARDWARE* DAS PLACAS

Este apêndice apresenta os perfis de *hardware* do ESP32 DevKit V1 e do Arduino Mega 2560 na forma em que são enviados ao modelo, dentro da mensagem de sistema de cada requisição. O arquivo completo de cada perfil contém ainda a disposição dos pinos na placa, a placa a selecionar na Arduino IDE e as instruções de ligação de cada componente, informações usadas apenas pela interface e omitidas do *prompt*. Cada perfil é enviado ao modelo em uma única linha; as quebras de linha abaixo foram acrescentadas para facilitar a leitura. No Mega, o número de um pino é o usado no código, de modo que o 54 corresponde à entrada analógica A0.

A.1 ESP32 DevKit V1

```
{
  "id": "esp32",
  "name": "ESP32 DevKit V1",
  "microcontroller": "ESP32 (dual-core Xtensa LX6, 240 MHz, WiFi e Bluetooth)",
  "language": "C++",
  "framework": "Arduino (núcleo ESP32)",
  "components": [
    {"name": "LED embutido", "type": "saída digital",
     "pins": [{"gpio": 2, "role": null}]},
    {"name": "LED verde", "type": "saída digital",
     "pins": [{"gpio": 25, "role": null}]},
    {"name": "LED amarelo", "type": "saída digital",
     "pins": [{"gpio": 26, "role": null}]},
    {"name": "LED vermelho", "type": "saída digital",
     "pins": [{"gpio": 27, "role": null}]},
    {"name": "Botão", "type": "entrada digital com pull-up interno",
     "pins": [{"gpio": 4, "role": null}]},
    {"name": "Buzzer", "type": "buzzer passivo, saída PWM",
     "pins": [{"gpio": 18, "role": null}]},
    {"name": "Sensor DHT22", "type": "sensor de temperatura e umidade, digital de um
        fio",
     "pins": [{"gpio": 15, "role": null}]},
    {"name": "Potenciômetro", "type": "entrada analógica (ADC de 12 bits, 0 a 4095)",
     "pins": [{"gpio": 34, "role": null}]},
    {"name": "Display OLED", "type": "SSD1306 128x64, I2C, endereço 0x3C",
     "pins": [{"gpio": 21, "role": "SDA"}, {"gpio": 22, "role": "SCL"}]},
    {"name": "Relé", "type": "módulo relé, saída digital ativa em nível alto",
     "pins": [{"gpio": 23, "role": null}]}
  ],
  "notes": [
    "Os GPIO 34 a 39 são somente entrada e não têm pull-up nem pull-down internos.",
    "Os pinos do ADC2 não leem valores analógicos com o WiFi ligado; entradas analó
        gicas usam pinos do ADC1 (GPIO 32 a 39).",
```

```
    "Se o estudante pedir para trocar um componente de pino, mantenha o nome da
       constante e use um pino livre: GPIO 13, 16, 17, 19, 32 ou 33. Evite os GPIO 5,
       12, 14 e 15, que definem o modo de boot ou mudam de estado ao ligar a placa.",
    "Use as bibliotecas Adafruit SSD1306 e Adafruit GFX para o display. Com I2C, inicie
        o barramento com Wire.begin(SDA_PIN, SCL_PIN), passando os pinos do display.",
    "Use a biblioteca DHT sensor library, da Adafruit, para o DHT22: crie o sensor com
       DHT dht(pino, DHT22), e readTemperature() já devolve graus Celsius.",
    "Para PWM, use analogWrite(pino, valor); para o buzzer, tone(pino, frequência) e
       noTone(pino). Não use ledcSetup nem ledcAttachPin, que não existem no núcleo
       ESP32 3.x.",
    "O monitor serial usa 115200 baud."
  ]
}
```

A.2 Arduino Mega 2560

```
{
  "id": "arduino-mega-2560",
  "name": "Arduino Mega 2560",
  "microcontroller": "ATmega2560 (8 bits, 16 MHz, 8 KB de RAM)",
  "language": "C++",
  "framework": "Arduino (núcleo AVR)",
  "components": [
    {"name": "LED embutido", "type": "saída digital",
     "pins": [{"gpio": 13, "role": null}]},
    {"name": "LED verde", "type": "saída digital",
     "pins": [{"gpio": 5, "role": null}]},
    {"name": "LED amarelo", "type": "saída digital",
     "pins": [{"gpio": 6, "role": null}]},
    {"name": "LED vermelho", "type": "saída digital",
     "pins": [{"gpio": 9, "role": null}]},
    {"name": "Botão", "type": "entrada digital com pull-up interno",
     "pins": [{"gpio": 2, "role": null}]},
    {"name": "Buzzer", "type": "buzzer passivo, tocado com tone()",
     "pins": [{"gpio": 8, "role": null}]},
    {"name": "Sensor DHT22", "type": "sensor de temperatura e umidade, digital de um
       fio",
     "pins": [{"gpio": 7, "role": null}]},
    {"name": "Potenciômetro", "type": "entrada analógica (ADC de 10 bits, 0 a 1023)",
     "pins": [{"gpio": 54, "role": null}]},
    {"name": "Display OLED", "type": "SSD1306 128x64, I2C, endereço 0x3C",
     "pins": [{"gpio": 20, "role": "SDA"}, {"gpio": 21, "role": "SCL"}]},
    {"name": "Relé", "type": "módulo relé de 5 V, saída digital ativa em nível alto",
     "pins": [{"gpio": 4, "role": null}]}
  ],
  "notes": [
```

```
    "Os pinos digitais funcionam em 5 V e fornecem até 20 mA cada.",
    "PWM (analogWrite) nos pinos 2 a 13 e 44 a 46.",
    "As entradas analógicas A0 a A15 leem de 0 a 1023 (10 bits, com referência de 5 V)
        .",
    "O I2C fica fixo nos pinos 20 (SDA) e 21 (SCL): inicie com Wire.begin(), sem
        argumentos.",
    "Os pinos 0 e 1 são a serial do USB, usada por Serial; não ligue componentes neles.
         A placa tem também Serial1 (pinos 18 e 19), Serial2 (16 e 17) e Serial3 (14 e
        15).",
    "Use a biblioteca DHT sensor library, da Adafruit, para o DHT22: crie o sensor com
        DHT dht(pino, DHT22), e readTemperature() já devolve graus Celsius.",
    "Use as bibliotecas Adafruit SSD1306 e Adafruit GFX para o display.",
    "O monitor serial usa 9600 baud.",
    "Se o estudante pedir para trocar um componente de pino, mantenha o nome da
        constante e use um pino livre: D3, D10 a D12, D22 a D53 (no conector duplo) ou
        A1 a A15."
  ]
}
```

APÊNDICE B — PEDIDOS USADOS NA AVALIAÇÃO

Este apêndice reproduz o texto exato enviado à plataforma em cada atividade do Quadro 3. Nas atividades A8 a A10, o pedido termina com a frase “Use as constantes WIFI_SSID e WIFI_PASSWORD para o nome e a senha da rede.”, omitida abaixo.

A1 Pisque o LED embutido a cada segundo.

A2 Faça o LED verde acender e apagar aos poucos, sem parar, variando o brilho com PWM.

A3 Acenda o LED vermelho enquanto o botão estiver pressionado e apague quando ele for solto.

A4 Toque no buzzer as notas dó, ré, mi, fá e sol, meio segundo cada, e repita a sequência a cada três segundos.

A5 Mostre no monitor serial a temperatura do DHT22 em graus Celsius a cada dois segundos.

A6 Mostre no display OLED a temperatura e a umidade do DHT22, atualizando a cada dois segundos.

A7 Divida o curso do potenciômetro em três faixas e acenda só o LED verde na primeira, só o amarelo na segunda e só o vermelho na terceira.

A8 Conecte a placa à rede WiFi e mostre no monitor serial o endereço IP obtido.

A9 Crie uma página web servida pela placa que mostre a temperatura e a umidade do DHT22.

A10 Crie um servidor web na placa em que o endereço /ligar liga o relé e /desligar o desliga.